Pairwise vs. Multi-Preference Optimization

Setup: One Prompt, Several Responses

Suppose we have one prompt:

Prompt: Explain why the sky is blue.

The model generates several candidate answers, each with a reward score:

Response	Reward
A	9
B	8
C	5
D	2

The question is: **how do we turn this ranked list into a training signal?** The two approaches below answer this differently.

Pairwise Preference Optimization

Basic Idea

In pairwise preference optimization, training uses **two** responses at a time:

$$(x, y_w, y_l)$$

where y_w is the preferred/winner response and y_l is the rejected/loser response. This is the basic setup behind methods such as **DPO**: for each training example, you have one chosen response and one rejected response.

Example

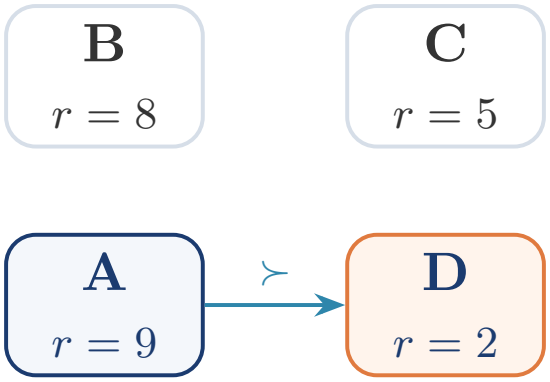
Picking a single pair

From our four responses, we might pick:

$$A \succ D.$$

The training signal is simply:

"Make A more likely than D."



The Limitation

We started with four responses, A, B, C, D, but the optimization step only directly compared A **vs.** D. Information about B and C — both of which carry useful signal about response quality — can be lost.

Multi-Preference Optimization

Basic Idea

Multi-preference optimization asks a natural question:

"Why compare only one winner and one loser when we have many responses with different quality?"

Instead of a single pair, we consider a **group** of responses on each side.

Example

Forming preference sets

From the same four responses, we can form:

$$Y^+ = \{A, B\} \qquad \text{and} \qquad Y^- = \{C, D\}.$$

Optimization now learns something closer to

$$\{A, B\} \text{ preferred over } \{C, D\},$$

rather than only $A \succ D$.